Personalized for: Professional Developer

■ Profile Snapshot

- ■ **Role:** Employed Professional — operating in fast-paced development environments with real deliverables and deadlines.
- ■ **Usage Pattern:** Daily user (1–2 hours/day) — Claude is a core productivity tool, not an experiment.
- ■ **Primary Needs:** Coding Help & Debugging + Deep Research & Summarization — dual-mode usage demanding both precision and depth.

■ Use Case Analysis

- ■ **Code Generation & Review:** Writing boilerplate, reviewing PRs, refactoring legacy code — needs accurate, runnable outputs with explanations.
- ■ **Debugging & Error Resolution:** Stack traces, logic bugs, runtime errors — needs fast, focused, contextual diagnosis.
- ■ **Technical Research:** Evaluating libraries, comparing architectures, understanding RFCs/docs — needs synthesis over raw data.
- ■ **Documentation & Summaries:** Summarizing complex repos, writing READMEs, condensing long technical articles for quick decisions.

■ Recommended Primary Model

Claude Sonnet 4.6

Your ideal daily driver. Sonnet 4.6 delivers near-Opus reasoning quality at significantly faster speeds — perfect for professional coding tasks and research synthesis where both accuracy and turnaround time matter. It handles complex multi-file debugging, architecture discussions, and long-document summarization without breaking a sweat.

- ■ Handles multi-thousand-line codebases and deep technical context without losing coherence.
- ■ Fast enough for interactive coding sessions — responses arrive before context switches break your flow.
- ■ Research synthesis quality rivals Opus for most technical domains (papers, docs, architecture tradeoffs).
- ■ Cost-effective for daily 1–2 hour usage — sustainable without burning through API credits.

■ Model Selection Guide

Claude Haiku 4.5

FAST & LIG HTWEIGHT

- ■ Quick syntax lookups — "What's the Python equivalent of Array.map?"
- ■ Simple regex generation or format conversion tasks.
- ■ Git command lookups, CLI flags, one-liners.
- ■ Explaining a single short function or config snippet.
- ■ Drafting quick Slack/email replies about technical decisions.

Claude Sonnet 4.6

DAILY DRIVER ★ R ECOMMEND ED

- ■ Debugging multi-file issues with stack traces and code context.
- ■ Writing full features, classes, or modules from spec.
- ■ Summarizing long technical docs, RFCs, or research papers.
- ■ Code reviews — spotting patterns, anti-patterns, security issues.
- ■ Comparing frameworks/libraries with pros, cons, and benchmarks.
- ■ Your default: 80–85% of all daily professional tasks.

Claude Opus 4.6

HEAVY LIFTING

- ■ Designing system architecture for complex, high-stakes projects.
- ■ Deeply ambiguous bugs with no obvious root cause after Sonnet attempts.
- ■ Strategic technical decisions: monolith vs microservices, DB selection.
- ■ Novel algorithm design or mathematical reasoning-heavy problems.
- ■ Synthesizing insights from 10+ sources into a coherent decision brief.

■■ Effort Level Guide

	■ Low	■ Standard	■ High	■ Max
When:	Quick lookups, syntax help, forma questions	Daily coding, debugging, doc summaries	Complex bugs, architecture reviews, research briefs	Critical architecture, novel algorithm design
Output:	Direct answers, short code snippets, one-liners	Full functions, explanations, structured summaries	Deep analysis, multiple options, trade-off tables	Exhaustive reasoning; slower — use selectively

■ ***Rule of thumb:*** *Start at Standard. Upgrade to High when Sonnet's first answer feels shallow or incomplete. Reserve Max for decisions you can't easily undo (architecture choices, production migrations).*

■■ Personalized Daily Workflow

1. **Morning Standup Prep (5 min)**
   Paste yesterday's unresolved issues → Sonnet Standard → get a crisp summary + suggested approach for today.
2. **Feature / Task Kickoff**
   Give Sonnet your ticket description + relevant file snippets → generate scaffold code, identify edge cases, suggest test cases.
3. **Active Coding Sessions**
   Use Sonnet as your pair programmer. Paste errors inline. Ask 'why does this fail?' and 'what's the idiomatic fix?'
4. **Mid-Session Deep Dives**
   Hit a wall? Switch to High effort or Opus. Share full context, ask for root cause analysis with multiple hypotheses.
5. **Research Blocks (30 min)**
   Paste documentation, GitHub issues, or paper abstracts → Sonnet High → get a structured summary with decision-ready insights.
6. **Code Review & Refactor**
   Paste your PR diff → Sonnet Standard → identify bugs, style violations, security issues, and suggested improvements.
7. **End-of-Day Documentation**
   Paste code you wrote today → Haiku or Sonnet → generate docstrings, README sections, inline comments.
8. **Weekly Architecture Review**
   Once a week: bring a design challenge to Opus Max for deep strategic analysis and long-term recommendations.

■ Quick Reference: Task → Model → Effort

Task	Best Model	Effort	Reason
Fix a runtime error / exception	Sonnet 4.6	Standard	Needs context awareness + speed
Write a new feature from a ticket	Sonnet 4.6	Standard	Balanced quality and turnaround
Design system architecture	Opus 4.6	Max	High-stakes, requires deep reasoning
Summarize a 50-page technical doc	Sonnet 4.6	High	Depth needed for accurate synthesis
Quick syntax / API lookup	Haiku 4.5	Low	Fast, simple, no deep reasoning needed
PR code review (full diff)	Sonnet 4.6	Standard	Pattern recognition at scale
Compare two libraries/frameworks	Sonnet 4.6	High	Nuanced trade-off analysis required
Novel algorithm / math-heavy problem	Opus 4.6	Max	Maximum reasoning capacity needed

■■ Biggest Mistakes to Avoid

- ■ **Using Opus for everything** — Overkill for 80% of tasks; slows your workflow and burns credits unnecessarily. Reserve it for genuinely hard problems.
- ■ **Vague prompts like 'fix my code'** — Always include: language, framework version, error message, what you tried, and expected vs actual behavior.
- ■ **Pasting code without context** — Share the surrounding functions, imports, and data structures. Claude can't debug in a vacuum.
- ■ **Accepting the first answer blindly** — For production code, ask for edge cases, failure modes, and unit tests alongside the solution.

■ **Never switching effort levels** — If Sonnet at Standard gives a shallow answer twice, bump to High or try Opus. Don't retry the same prompt expecting different depth.

■ Final Recommendation

ONE MODEL	ONE EFFORT
Claude Sonnet 4.6	**Standard Effort**

As a professional developer using Claude daily for coding and research, **Sonnet 4.6 at Standard effort** is your optimal default for 80–85% of all tasks. It delivers the right balance of reasoning depth, code accuracy, and response speed to keep you in flow without context-switching overhead. It handles everything from complex bug diagnosis to multi-document research synthesis — the exact two pillars of your daily work. Upgrade to **High effort** when a problem is genuinely ambiguous, and reach for **Opus at Max** only for architectural decisions or novel problems that warrant the extra depth. Use **Haiku** for the quick lookups that don't need a full response.